Disclaimer: These slides can include material from different sources. I'll be happy to explicitly acknowledge a source if required. Contact me for requests.

Machine Learning in a Nutshell

15-288 Spring '26

Lecture 18:
Support Vector Machines

Teacher:
Gianni A. Di Caro

جامعة كارنيجي ميلون في قطر
Carnegie Mellon University Qatar

At Pittsburgh G20 summit …

Disclaimer: The following slides present *some* mathematical details about SVMs, kernels and kernelization. The mathematical details are given for completeness and to be able to follow the logic of the discourse, however, it is enough to grab the basic ideas here.

Our fundamental questions about linear classifiers

$$f_{\boldsymbol{w}}(\boldsymbol{x}) = w_1 x_1 + w_2 x_2 + \cdots + w_d\, x_d + w_0$$

$$f_{\boldsymbol{w}}(\boldsymbol{x}; \Phi) = w_1 \phi_1(\boldsymbol{x}) + w_2 \phi_2(\boldsymbol{x}) + \cdots + w_m\, \phi_m(\boldsymbol{x}) + w_0$$

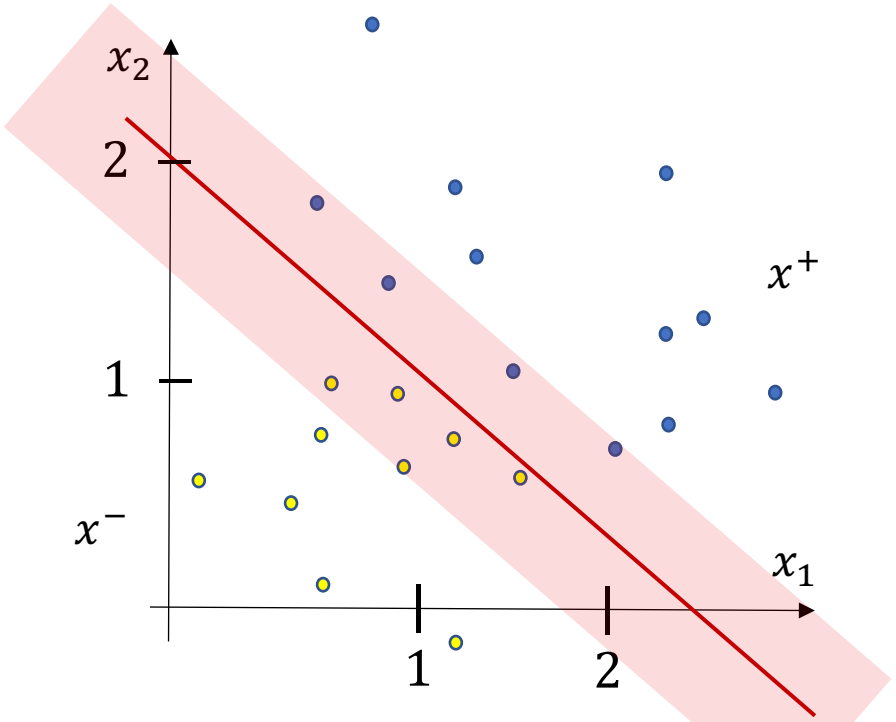
- ✓ How do we use a linear classifier to **make predictions**?
  - Which **classes** correspond to the values $1, -\frac{5}{2}, 99, \ldots$?
- ✓ **Given a loss function** ℓ, how do we **train / learn** a linear classifier?
  - ✓ How do we find the ***best*** linear decision boundary?
    - ➢ The ***best***?
    - ➢ How do we decide about which loss ℓ to use?
      - ✓ We know that different losses result in different boundaries

Where's the *critical region* for the classifier?

❖ *Given* ***linearly separable data:***

✓ We expect that a classifier doesn't make mistakes for input data that lie **far away from the linear separator** (both sides)

➢ Classification errors might occur for inputs that lie ***about the linear separator*** *(both sides)*



$$f_{\boldsymbol{w}}(\boldsymbol{x}) = \frac{3}{4}x_1 + x_2 - 2 = 0$$

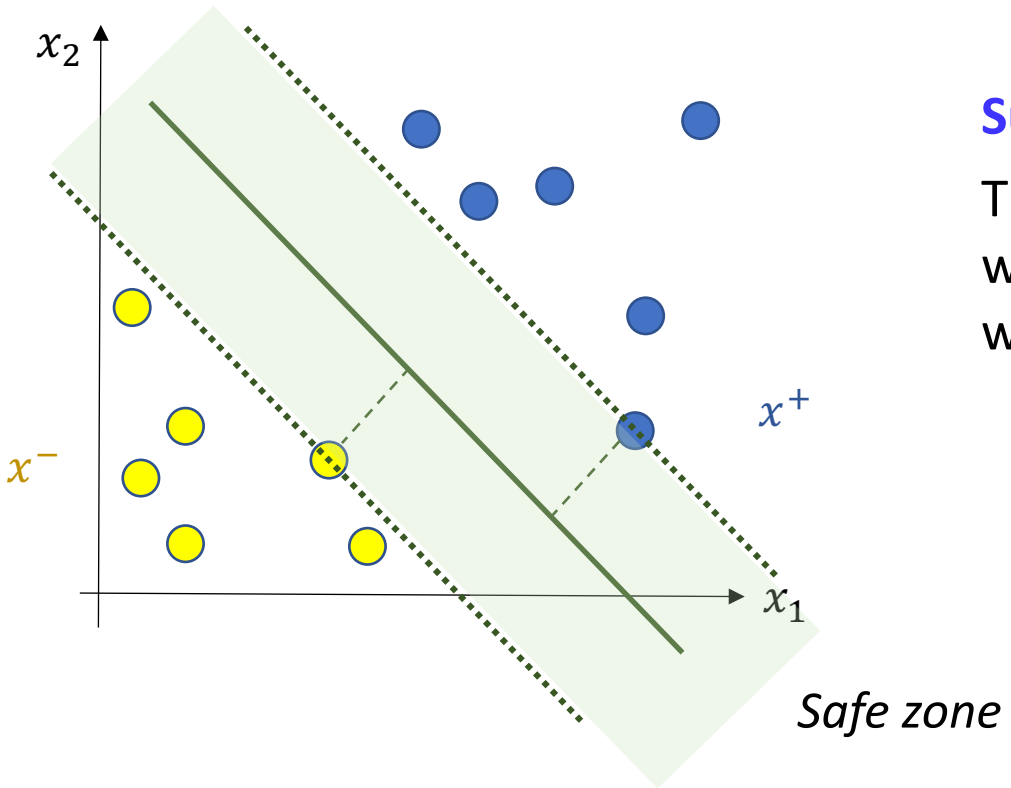
We know that we can *quantify* this expected error rate / confidence of the classifier using the notion of **score**

Classifier margin and choice of the best linear separator

Which linear classifier is *the best* (i.e., is most likely to generalize well), given the data?

- ✓ The one whose line/hyperplane **minimizes the error** on the training set (*consistent*)
- ✓ The one whose line/hyperplane **maximizes the classifier margin** for the two classes

❖ **Classifier margin**: distance of closest examples from the line/hyperplane



Support Vector Machine (SVM):

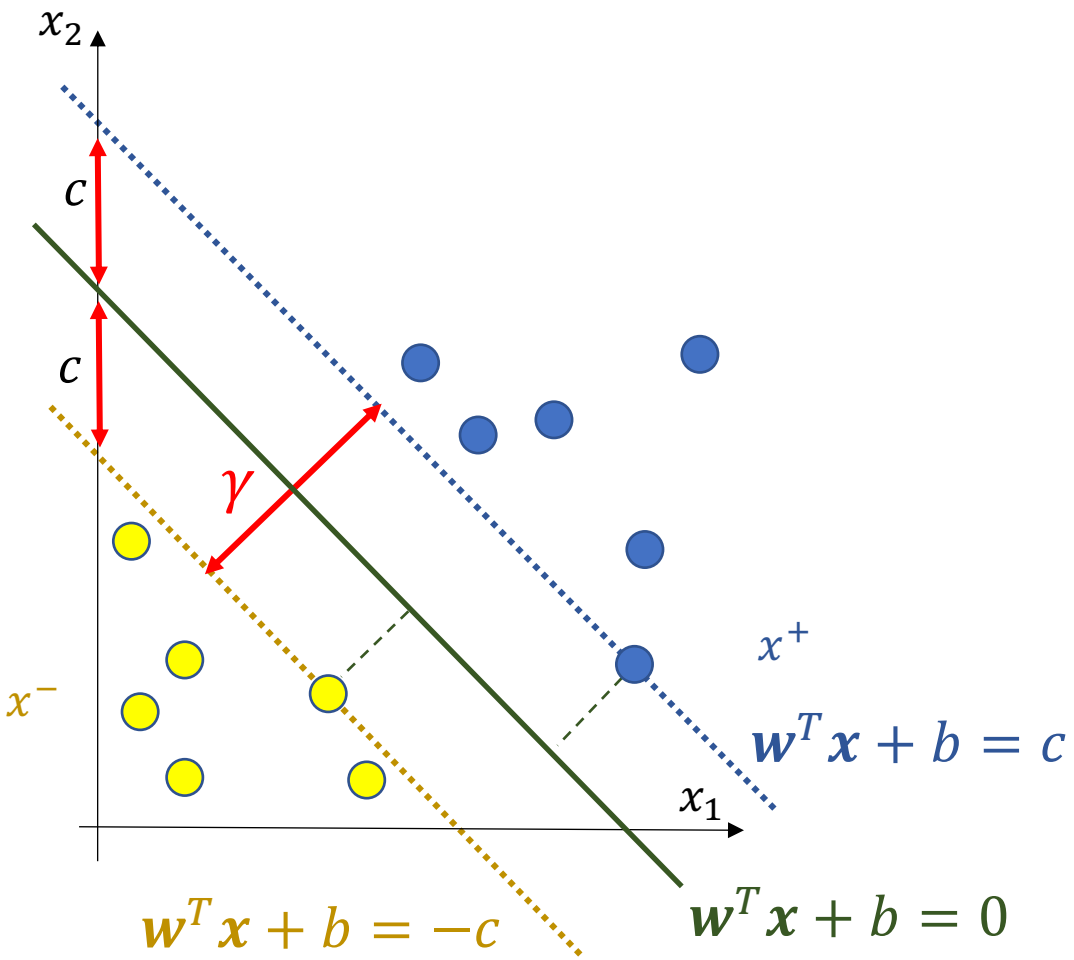
The **Linear classifier** selecting the linear separator with the objective of maximizing classifier margin while minimizing the error on the training set

Max-Margin Classifier

Classifier margin

- **Classifier margin, γ**: *distance* between the closest examples from the two classes w.r.t. the line/hyperplane

Classifier margin = $\gamma = \frac{2c}{\|\boldsymbol{w}\|}$

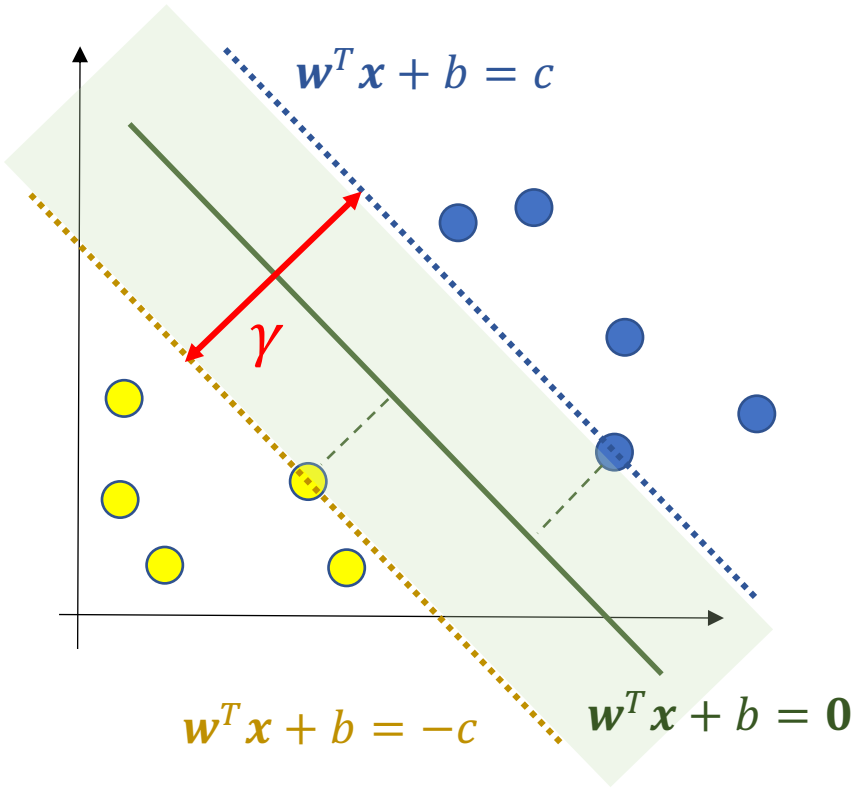


- $h_{\boldsymbol{w}}(\boldsymbol{x}) = \boldsymbol{w}^T\ \boldsymbol{x} + b = 0$ is the **linear separator**
- $\boldsymbol{w}^T\boldsymbol{x} + b = 0$ passes at an **equal distance** from the closest example(s) from each class
- Margin: $h_{\boldsymbol{w}}(\boldsymbol{x})$ defines **two other lines, parallel to it**, <u>one from each side</u>, each passing through the closest example(s) from each one of the two classes
  - $\boldsymbol{w}^T\boldsymbol{x} + b = +c$
  - $\boldsymbol{w}^T\boldsymbol{x} + b = -c$

  c is the increase/decrease in the *intercept* of the two lines wrt $h_{\boldsymbol{w}}(\boldsymbol{x})$

SVM Goal: Maximize classifier margin (keeping consistency)

❖ **SVM optimization problem** <u>given linearly separable classes</u>



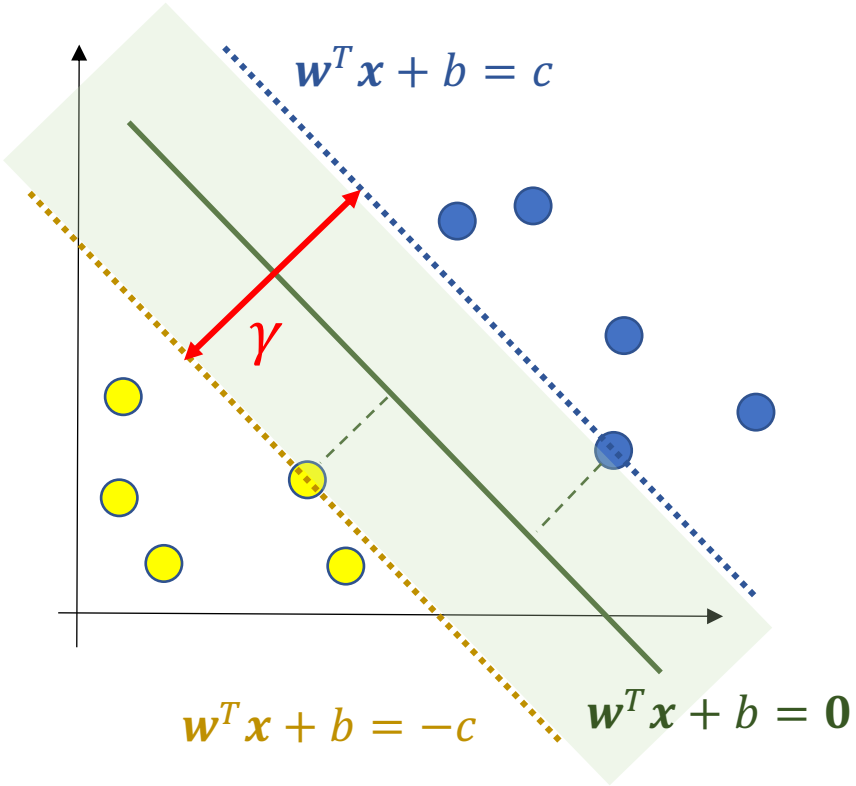
1. Find the linear separator $(\boldsymbol{w}, b)$ that **maximizes** the *classifier margin* $\gamma = \frac{2c}{\|\boldsymbol{w}\|}$

   → Favor **generalization:** minimize the expected error on test data in the **critical** region between the two classes

2. Subject to the **constraint** that all training examples $\boldsymbol{x}$ have a margin $(\boldsymbol{x}^T\boldsymbol{w} + b)y \geq c$

   → Satisfy **consistency**: all training examples are correctly classified (i.e., are outside of margin band)

SVM Constrained Optimization problem: formulation



$$\max_{\boldsymbol{w},b} \frac{2c}{\| \boldsymbol{w} \|}$$

Objective

Feasibility set / region

$$s.t. \qquad y^{(i)}\left(\boldsymbol{w}^T\boldsymbol{x}^{(i)} + b\right) \geq c, \qquad i = 1, \cdots, m$$

Each data point must be:

- on the correct side of the margin, i.e., $\geq c$
- and at least at a distance of $1/\|w\|$ away from the decision boundary, i.e., on the margin line (=) or more inside the region.

SVM optimization problem: a better formulation

- The (Euclidean) norm is a $\sqrt{\ }$ which is maybe not so nice to optimize and do calculations. On the other hand, the *square root is a monotonic function*, such that we can **get rid of the square root** without affecting the result of the $\max$ operation
  - ✓ Objective becomes: $\dfrac{2c}{\| \boldsymbol{w} \|^2}$

- The geometric margin is **scaling invariant** w.r.t. $\boldsymbol{w}$ such that we can safely operate the coordinate transformation $\boldsymbol{w}' = c \cdot \boldsymbol{w}$
  - ✓ Objective becomes: $\dfrac{2}{\| \boldsymbol{w} \|^2}$
  - ✓ Constraints become: $y^{(i)}\left(\boldsymbol{w}^T \boldsymbol{x}^{(i)} + b\right) \geq 1$

$$\max_{\boldsymbol{w},b} \frac{2c}{\| \boldsymbol{w} \|}$$

$$s.t. \quad y^{(i)}\left(\boldsymbol{w}^T \boldsymbol{x}^{(i)} + b\right) \geq c, \quad i = 1, \cdots, m$$

$$\rightarrow \quad \boxed{\begin{aligned} &\min_{\boldsymbol{w},b} \frac{\| \boldsymbol{w} \|^2}{2} \\ &s.t. \quad y^{(i)}\left(\boldsymbol{w}^T \boldsymbol{x}^{(i)} + b\right) \geq 1, \quad i = 1, \cdots, m \end{aligned}}$$

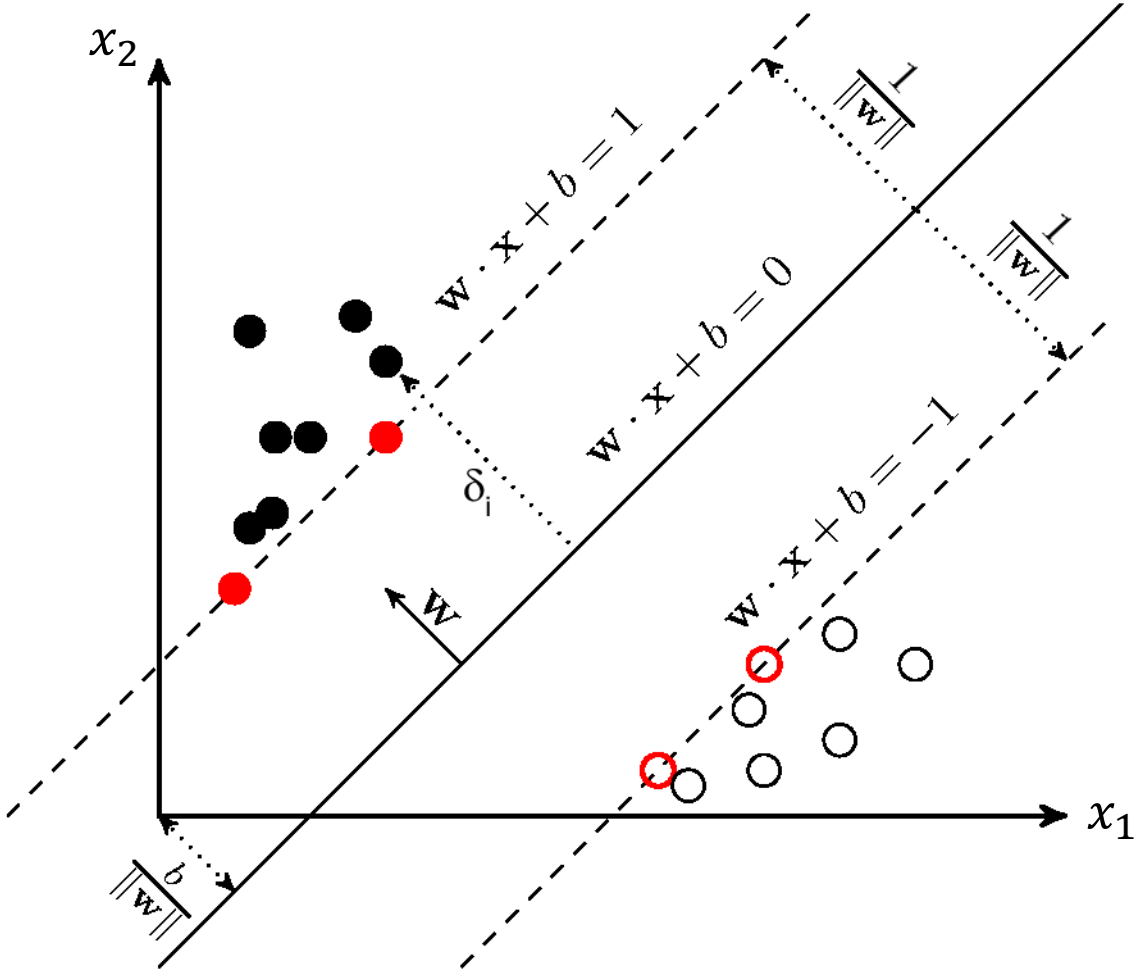
SVM (hard-margin) optimization problem linearly separable case

$$\min_{\boldsymbol{w},b} \frac{\| \boldsymbol{w} \|^2}{2}$$

$$s.t. \quad y^{(i)}\left(\boldsymbol{w}^T \boldsymbol{x}^{(i)} + b\right) \geq 1, \quad i = 1, \cdots, m$$

Quadratic (**convex**) optimization problem with m linear inequality constraints

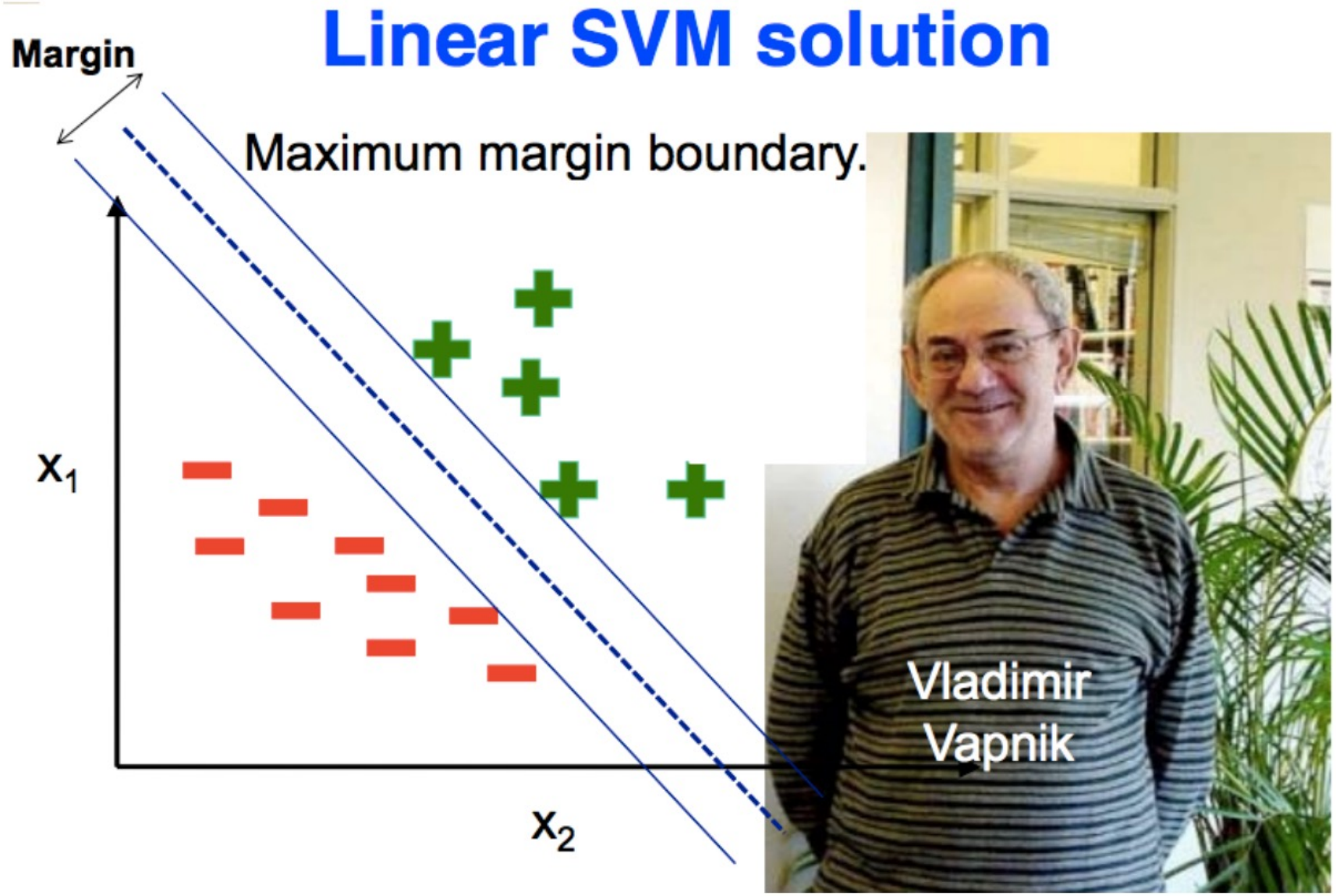
Large / difficult but solvable in **polynomial time**!



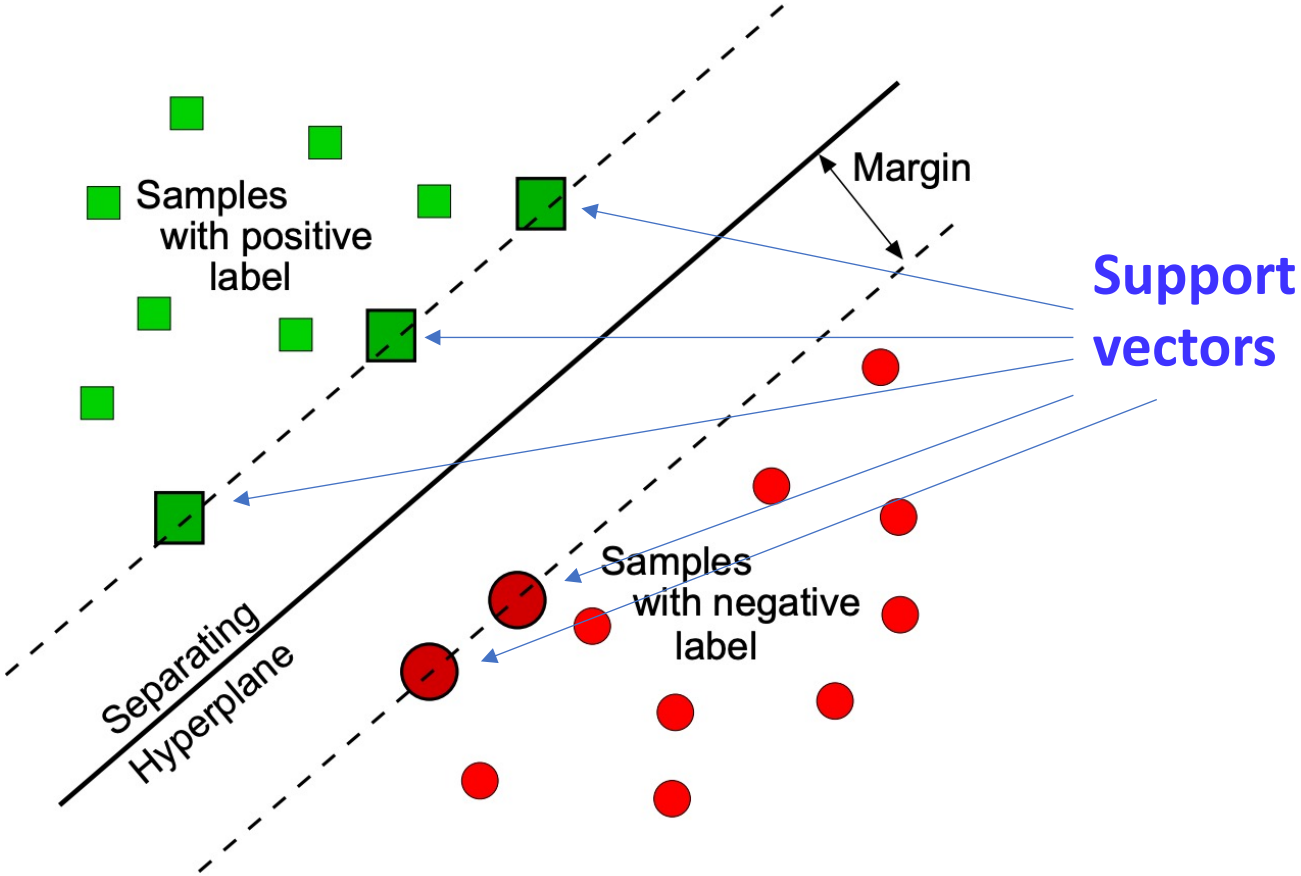
Equivalent forms for the objective:

$$\| \boldsymbol{w} \|^2 = \sum_{i=1}^{d} w_i^2 \qquad \| \boldsymbol{w} \|^2 = \boldsymbol{w}^T \cdot \boldsymbol{w}$$

SVMs (1963, 1993)



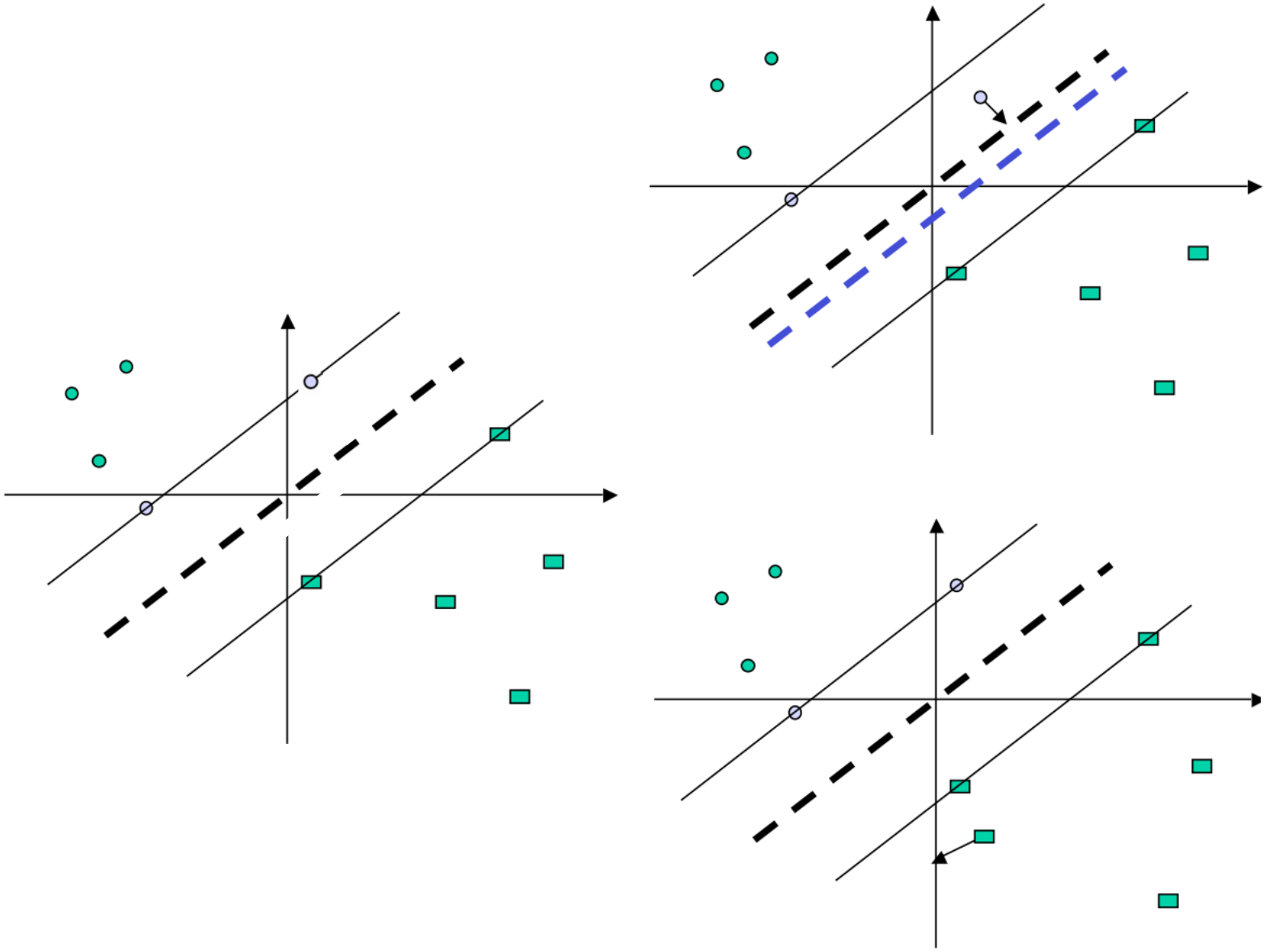
Support vectors



- The separating hyperplane is uniquely defined by a **small subset of training data**, those that are the nearest to the separating hyperplane, from both sides

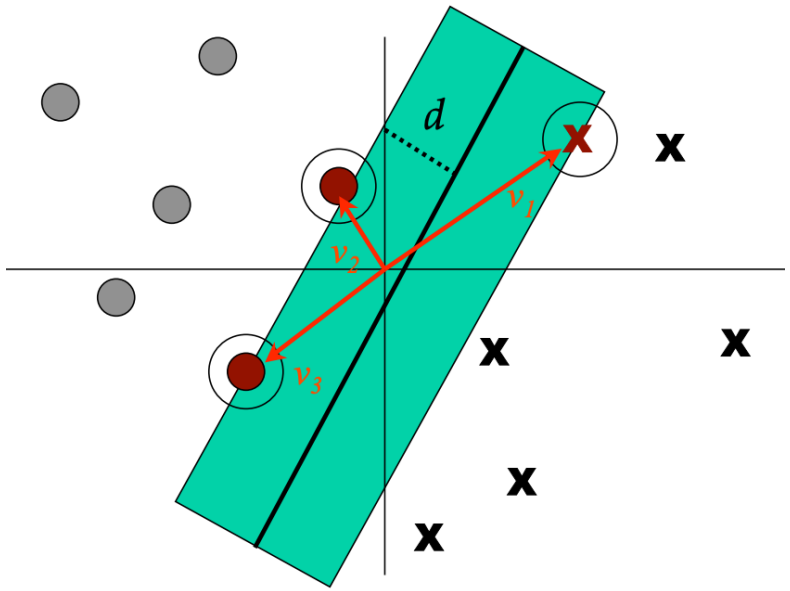
  - These points are called **support vectors**

Support vectors



- **Support vectors** are the critical points: removing / moving them would change the decision boundary
  - Removing / little moving other points doesn't affect the decision boundary

Support vectors



- For Support Vectors $\boldsymbol{x}^{(i)}$

$$y^{(i)}\left(\boldsymbol{w}^T\boldsymbol{x}^{(i)} + b\right) = 1$$

Why? They correspond to solutions where the **constraint in the opt problem is active**: inequality is satisfied with equality, the point is on the **margin line**

- The **weighted sum** of the Support Vectors is $\boldsymbol{w}$, the normal vector of the hyperplane:

$$\boldsymbol{w} = \sum_{\{i \mid \boldsymbol{x}^{(i)} \text{ is support vector}\}} \alpha_i \boldsymbol{x}^{(i)} y^{(i)}$$

- Since $\boldsymbol{w}$ is what we are looking for → <u>we only need the support vectors</u> to represent $\boldsymbol{w}$!

Solution of SVM for the linearly separable case: primal and dual

❖ Primal problem:

$$\min_{\boldsymbol{w},b} \frac{\boldsymbol{w}^T \cdot \boldsymbol{w}}{2}$$

$$s.t. \left(\boldsymbol{w}^T \boldsymbol{x}^{(j)} + b\right) y^{(j)} \geq 1 \quad j = 1, \cdots, m$$

m training points $(\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)} \cdots, \boldsymbol{x}^{(m)})$

d features $\boldsymbol{x}^{(i)} \in \mathbb{R}^d$

$\boldsymbol{w}$ – weights on features (d-dim problem)

❖ Dual problem:

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} \alpha_i \alpha_j y^{(i)} y^{(j)} \left(\boldsymbol{x}^{(i)} \cdot \boldsymbol{x}^{(j)}\right)$$

$$s.t. \sum_{i=1}^{m} \alpha_i y^{(i)} = 0$$

$$\alpha_i \geq 0, \qquad i = 1, \cdots, m$$

$\boldsymbol{\alpha}$ – weights (multipliers) on training pts (m-dim problem)

Solution of SVM for the linearly separable case

❖ Dual problem:

$\boldsymbol{\alpha}$ – weights (multipliers) on training pts (m-dim problem)

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} \alpha_i \alpha_j y^{(i)} y^{(j)} \left(\boldsymbol{x}^{(i)} \cdot \boldsymbol{x}^{(j)}\right)$$

$$s.t. \sum_{i=1}^{m} \alpha_i y^{(i)} = 0$$

$$\alpha_i \geq 0, \qquad i = 1, \cdots, m$$

❖ Solution of Dual problem:

Dual problem is also QP, solution gives $\boldsymbol{\alpha}$, that can be used to compute $(\boldsymbol{w}, b)$ and make **predictions**

Only support vectors matter!

$$\boldsymbol{w} = \sum_{i=1}^{m} \alpha_i \boldsymbol{x}^{(i)} y^{(i)}$$

Only the $\boldsymbol{x}^{(i)}$ that are support vectors have weight $\alpha_i > 0$

$$b = y^{(k)} - \boldsymbol{w} \cdot \boldsymbol{x}^{(k)}$$

Computed for any k where $\alpha_k > 0$

Dual SVM: Predictions

Prediction label, $y \in \{-1, +1\}$, for a new input $\boldsymbol{z}$:

$$\boxed{y(\boldsymbol{z}) = w_1 z_1 + w_2 z_2 + \cdots + w_d z_d + b}$$

$$\boldsymbol{w} = \sum_{i=1}^{m} \alpha_i \boldsymbol{x}^{(i)} y^{(i)}$$

Only the $\boldsymbol{x}^{(i)}$ that are support vectors have weight $\alpha_i > 0$

$$b = y^{(k)} - \boldsymbol{w} \cdot \boldsymbol{x}^{(k)}$$

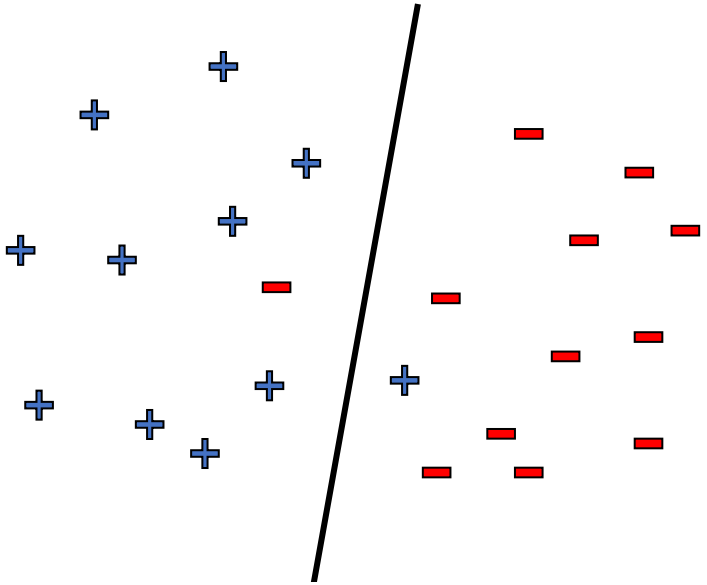
Computed for any k where $\alpha_k > 0$

$$y(\boldsymbol{z}) = sign(b + \boldsymbol{z}^T \cdot \boldsymbol{w}) = sign\left(b + \boldsymbol{z}^T \cdot \sum_i \alpha_i y_i \boldsymbol{x}_i\right)$$

$$= sign\left(y_k - \boldsymbol{w}^T \cdot \boldsymbol{x}_k + \boldsymbol{z}^T \cdot \sum_i \alpha_i y_i \boldsymbol{x}_i\right)$$

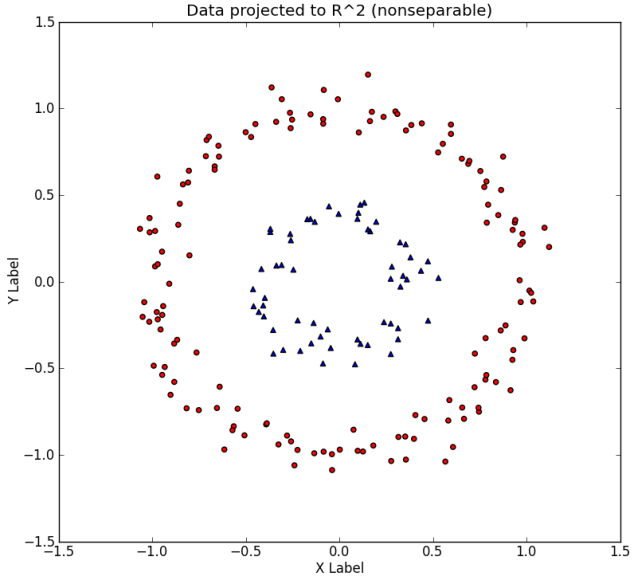
$$= sign\left(y_k - \sum_i \alpha_i y_i \boldsymbol{x}_i^T \cdot \boldsymbol{x}_k + \sum_i \alpha_i y_i \boldsymbol{z}^T \cdot \boldsymbol{x}_i\right)$$

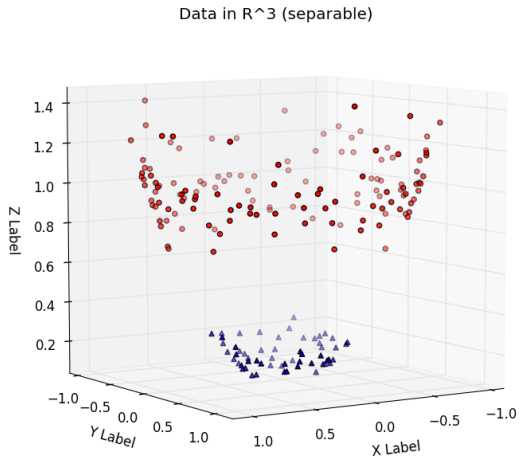
What if data is NOT linearly separable?



Use features of features of features of features….

$$x_1^2, x_2^2, x_1x_2, \cdots, \exp(x_1)$$

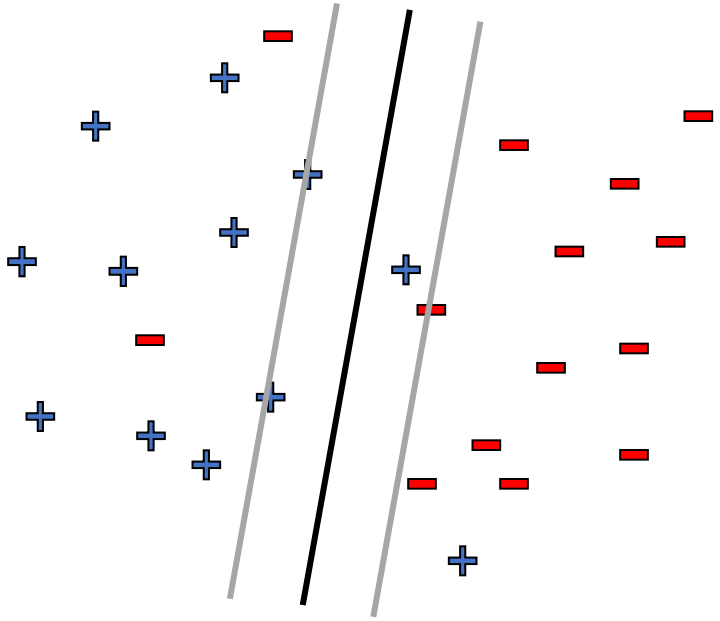




- Run risk of overfitting
- Computational issues
- How to find the right features …?

What if data is still not linearly separable?

❖ Allow *errors* in classification!



$$\min_{\boldsymbol{w},b} \frac{\boldsymbol{w}^T \cdot \boldsymbol{w}}{2}$$

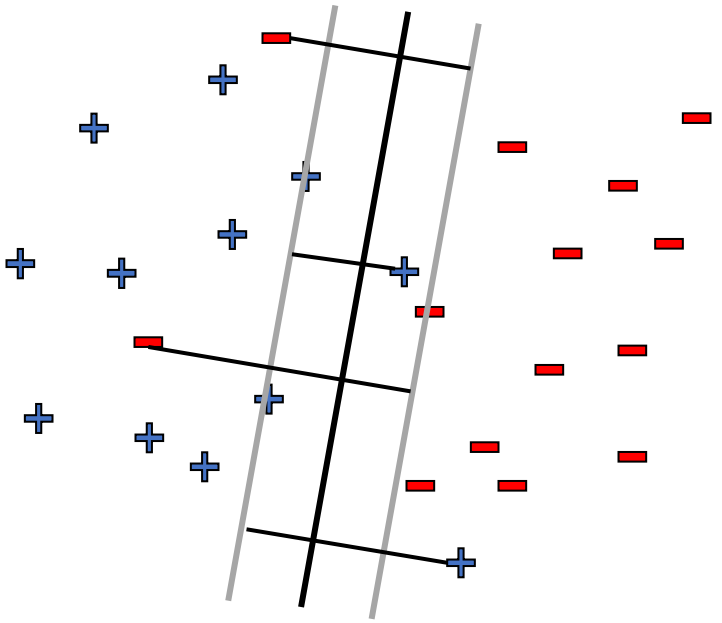
$$s.t. \left(\boldsymbol{w}^T \boldsymbol{x}^{(j)} + b\right) y^{(j)} \geq 1 \quad j = 1, \cdots, m$$

Can we reuse this as it is?

What if data is still not linearly separable? → Soft-Margin SVM

❖ Allow *error* in classification

✓ Pay a penalty p for errors!



✓ **Soft margins**: some examples are within the margin zone

✓ Some examples are misclassified

$$\min_{\boldsymbol{w},b,\xi} \ \boldsymbol{w}^T \cdot \boldsymbol{w} + p \sum_{j=1}^{m} \xi_j$$

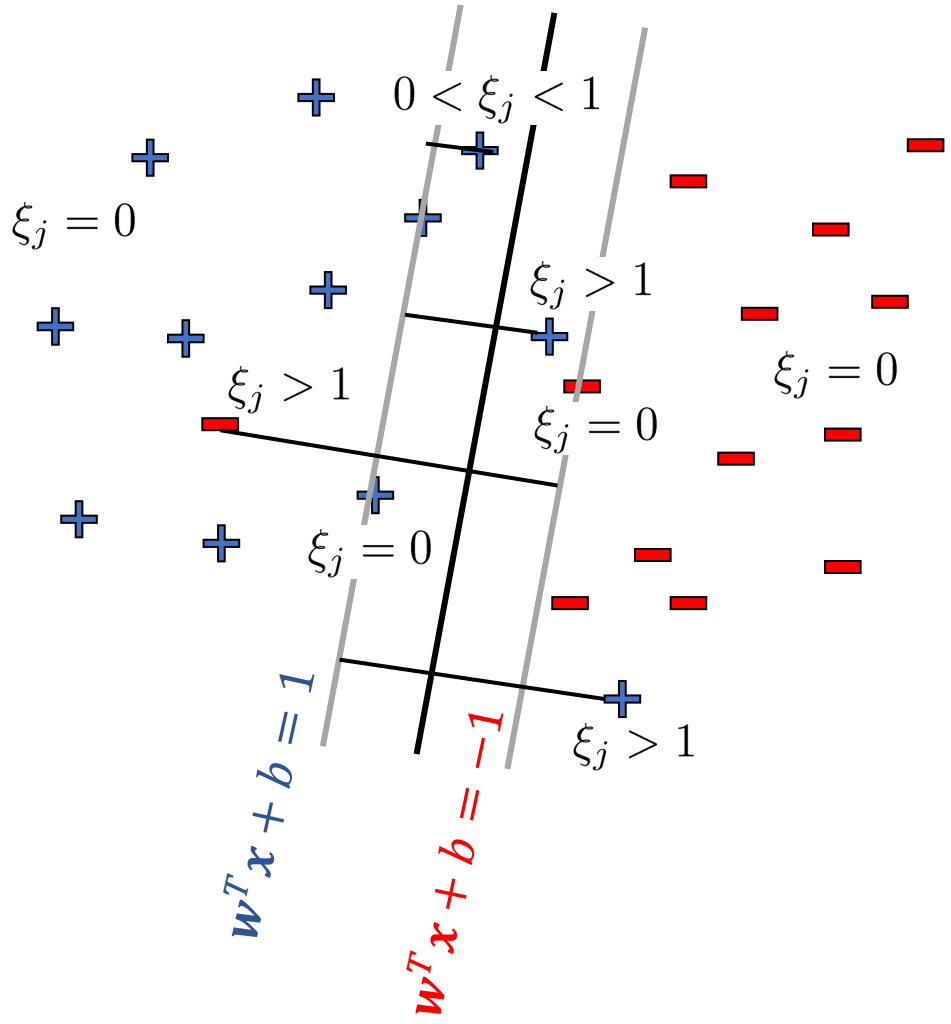
$$s.t. \quad y^{(i)}\left(\boldsymbol{w}^T \boldsymbol{x}^{(i)} + b\right) \geq 1 - \xi_i \qquad i = 1, \cdots, m$$

$$\xi_i \geq 0 \qquad i = 1, \cdots, m$$

- ξ_i = **slack variable**, gets a value > 1 if $\boldsymbol{x}^{(i)}$ is misclassified
- $0 < \xi_i < 1$ if $\boldsymbol{x}^{(i)}$ is correctly classified but within the margin
- $\xi_i = 0$ if $\boldsymbol{x}^{(i)}$ is correctly classified
- p penalty to pay for misclassifications and soft classifications

✓ Still a convex, quadratic programming problem ☺

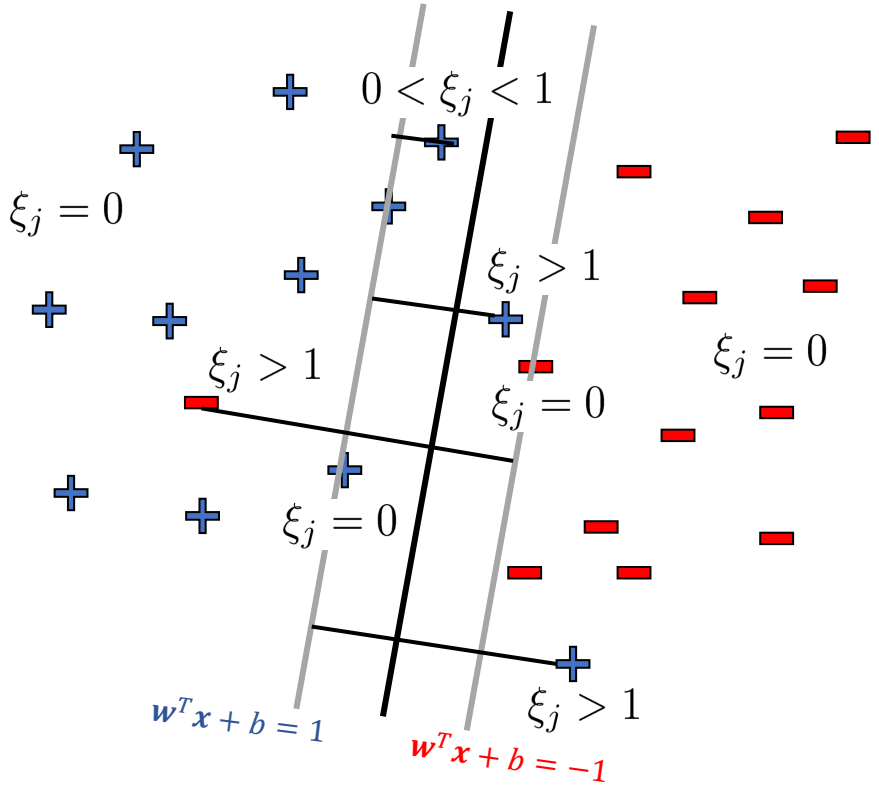
Slack variables

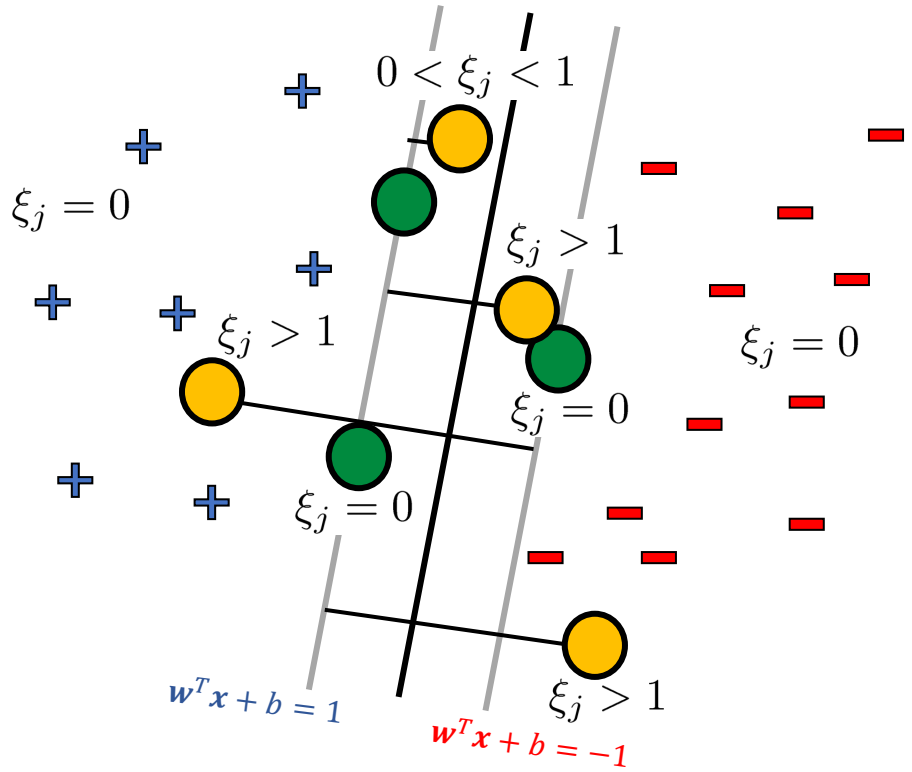


- $\xi_i > 1$ if $\boldsymbol{x}^{(i)}$ is misclassified
- $0 < \xi_i < 1$ if $\boldsymbol{x}^{(i)}$ is correctly classified, within margin
- $\xi_i = 0$ if $\boldsymbol{x}^{(i)}$ is correctly classified

- ξ_i is linearly proportional to the distance from the class margin if $\boldsymbol{x}^{(i)}$ is misclassified or within the margins
- In the obj <u>pay a linearly proportional penalty</u> for mistakes, a small one for being inside the margin
- p penalty for unit of distance mistake, trade-off parameter between **hard** and **soft** objectives (usually set by **cross-validation**, e.g., $p = 1/d$)

Support Vectors in soft-margin SVM





- **Margin support vectors**
  - ✓ $\xi_i = 0, \quad y^{(i)}\left(\boldsymbol{w}^T \boldsymbol{x}^{(i)} + b\right) = 1$
  - ➢ Don't contribute to objective but enforce constraints on solution
  - ➢ Correctly classified but on margin

- **Non-margin support vectors**
  - ✓ $\xi_i > 0$
  - ➢ Contribute to both objective and constraints
  - ○ $0 < \xi_i < 1$ Correctly classified but inside margin
  - ○ $\xi_i > 1$ Incorrectly classified

SVM dual optimization problems

$$\max_{\boldsymbol{\alpha}} \sum_i \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j \, y_i y_j \, \boldsymbol{x}_i^T \cdot \boldsymbol{x}_j$$

$$\sum_i \alpha_i y_i = 0$$

$$C \geq \alpha_i \geq 0, \qquad i = 1, \cdots, m$$

$C \to \infty$

Hard-margin SVM dual problem formulation

$$\max_{\boldsymbol{\alpha}} \sum_i \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j \, y_i y_j \, \boldsymbol{x}_i^T \cdot \boldsymbol{x}_j$$

$$\sum_i \alpha_i y_i = 0$$

$$\alpha_i \geq 0, \qquad i = 1, \cdots, m$$

Soft-margin SVM dual problem formulation

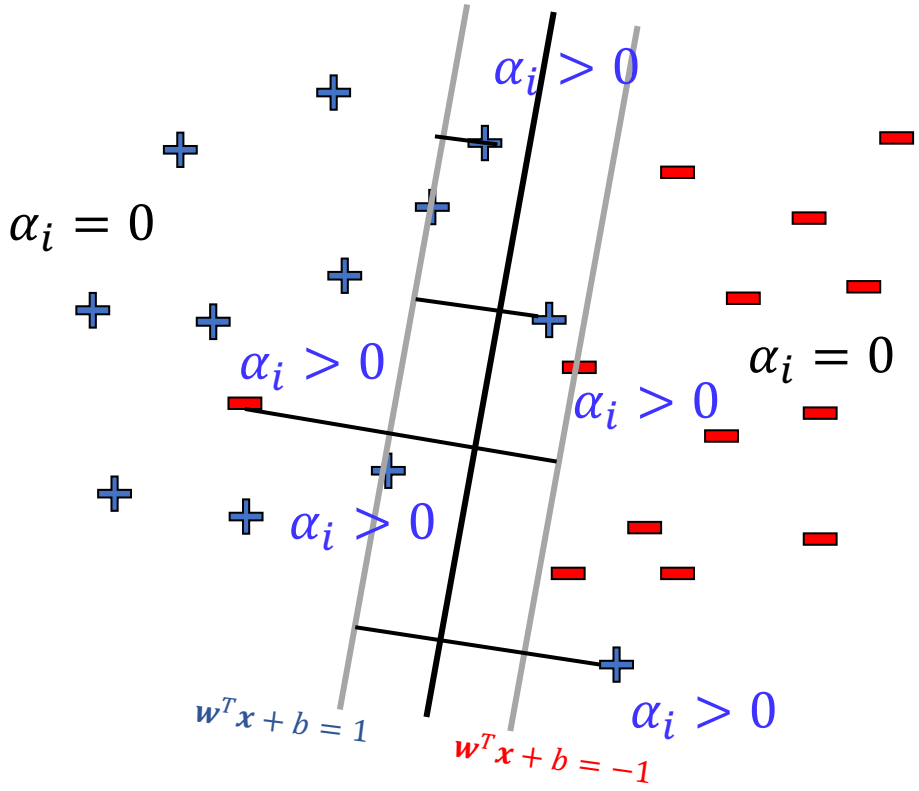
Variables of Dual SVM, $\boldsymbol{\alpha}$

$$\max_{\boldsymbol{\alpha}} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j \, y_i y_j \, \boldsymbol{x}_i \cdot \boldsymbol{x}_j$$

$$\sum_i \alpha_i y_i = 0$$

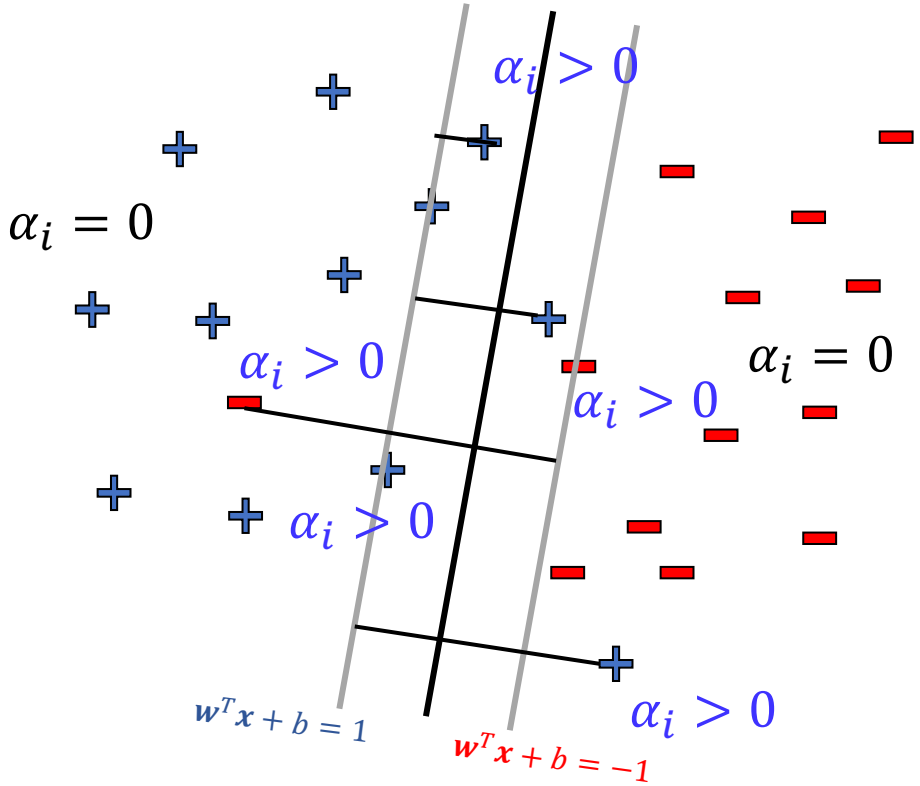
$$C \geq \alpha_i \geq 0, \qquad i = 1, \cdots, m$$

Solution of the quadratic dual problem gives α_i



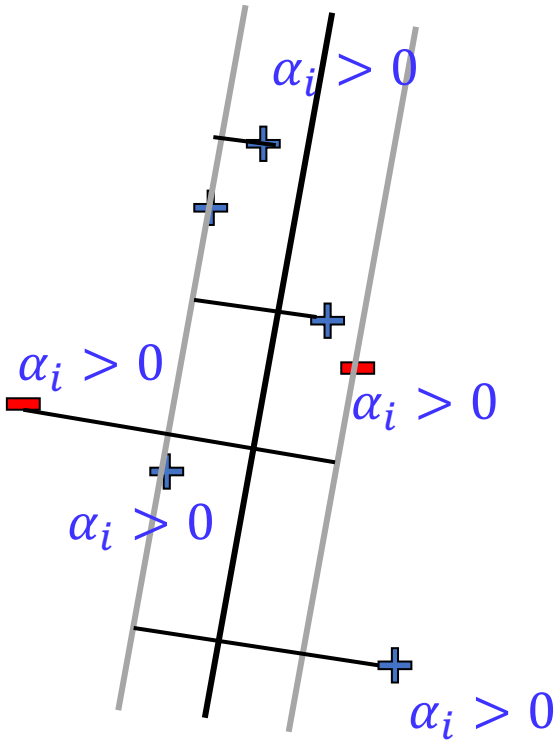
- For examples $\boldsymbol{x}_i$ that are **not support vectors** the corresponding multiplier α_i is zero
- For examples $\boldsymbol{x}_i$ that are **support vectors** the corresponding multiplier α_i is > 0

SVMs define an embedding



SVM: $D \rightarrow SV$

(Embedding)



- The SVM defines a mapping from an input matrix defined by the vectors in D (25 in the figure) to a much smaller matrix, defined by the subset of SV (7 in the figure), capturing latent relationships among the example vectors (e.g., examples with large positive margins are irrelevant for making a prediction *given* the SV).
- Each vector x_i in SV has weight, $\alpha_i \propto$ **Embedding strength** of example x_i for the classifier

Solution of Dual SVM

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j \, y_i y_j \, \boldsymbol{x}_i^T \cdot \boldsymbol{x}_j$$

$$\sum_i \alpha_i y_i = 0$$

$$C \geq \alpha_i \geq 0, \qquad i = 1, \cdots, m$$

- Linearly separable case: C disappears!

Solution of the quadratic dual problem gives α_i $\rightarrow$

$$\boldsymbol{w} = \sum_i \alpha_i y_i \boldsymbol{x}_i$$

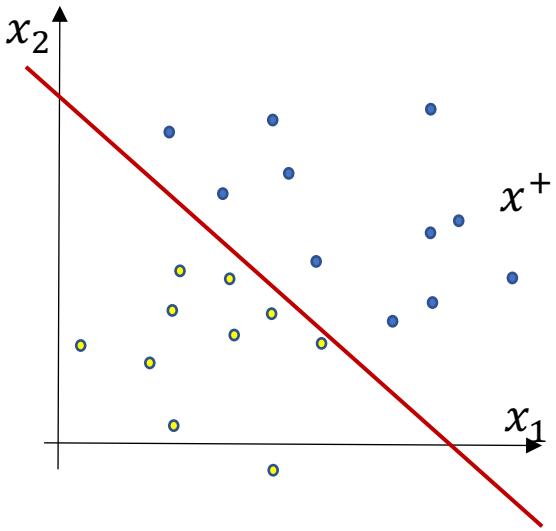
$$b = y_k - \boldsymbol{w}^T \cdot \boldsymbol{x}_k$$

computed for *any* k where $C > \alpha_k > 0$

- ❖ The vector $\boldsymbol{w}$ (the slope of the linear classifier) is defined as a linear combination of the example vectors $\boldsymbol{x}_i$
- ➢ The coefficients of the linear combination are the dual variables α_i
  - ✓ Only the support vectors in SV contribute to define $\boldsymbol{w}$!
  - ✓ Each example $\boldsymbol{x}_i$ contributes with a strength proportional to α_i

From Linear to Non-Linear SVMs

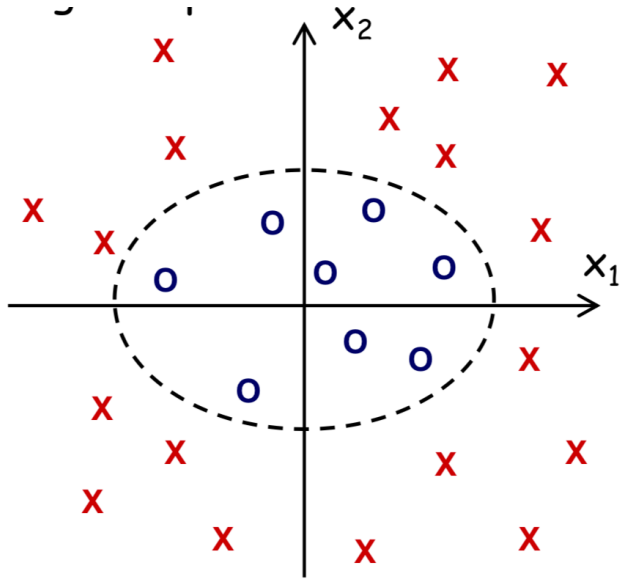
- So far, when data is **linearly separable** or nearly linear, SVMs are an excellent option!



- What about when data are **highly non-linearly separable?** Can we still use SVMs? Or other linear classifiers?

→ yes, using **Kernelization**!

(to be continued …)



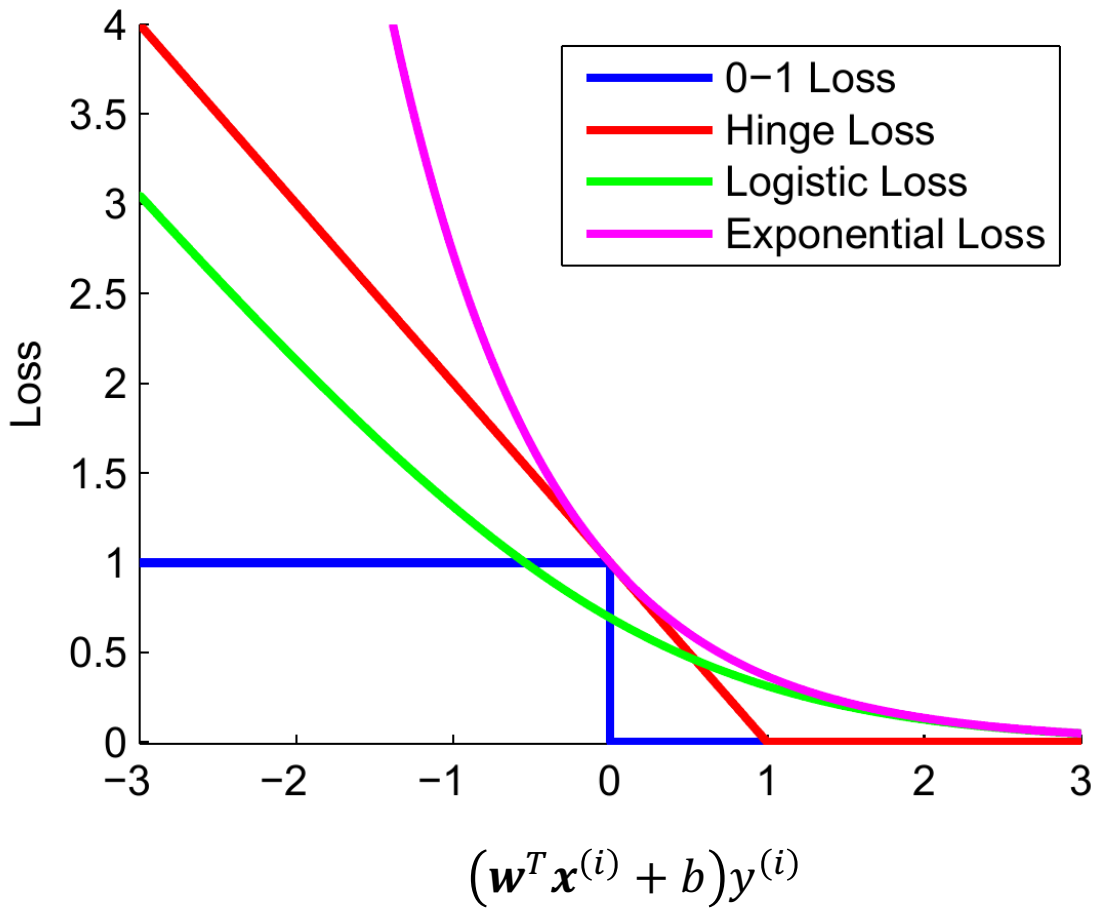
The following slides on the **hinge loss** are somewhat technical and are included for completeness. You are not expected to follow the details.

- The main concept that you should get is the following: training a linear classifier using the hinge loss (together with a regularization term) is equivalent to training a soft-margin SVM (i.e., the decision boundaries would be equivalent)

- In other words, the hinge loss captures the SVM principle of maximizing the margin while allowing for some violations (i.e., balancing margin maximization and classification consistency).

Hinge Loss and Loss Functions

- ✓ Intuition: hinge loss upper bounds 0-1 loss, has non-trivial gradient
- ✓ Loss = 0 only for margin at least 1
- ✓ Try to increase margin if less than 1:
  → **Max-margin classifier!**



Linear Classifier with Hinge Loss Function (SVMs)

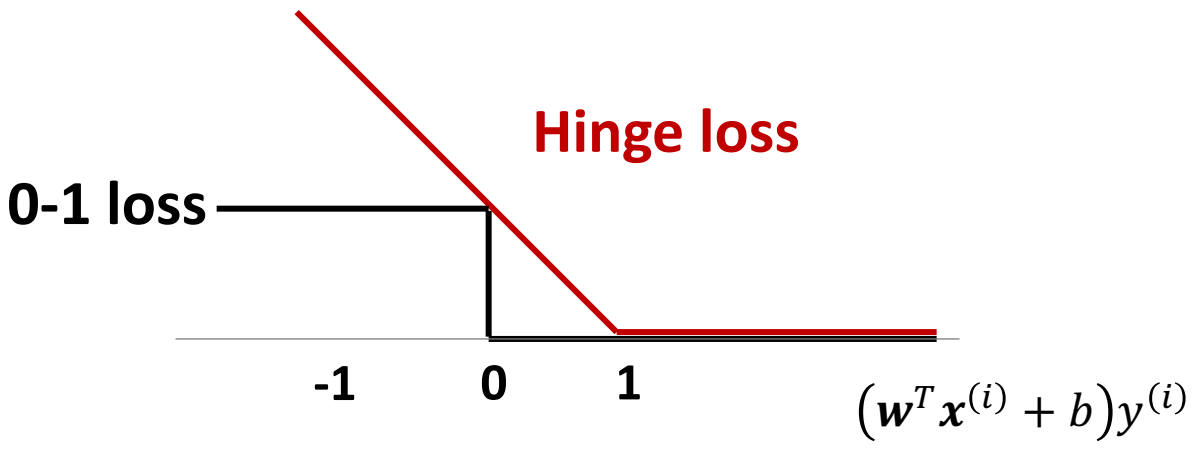
❖ Hinge loss optimization problem for a linear classifier :

$$h_{\boldsymbol{w}}(\boldsymbol{x}) = w_1 x_1 + w_2 x_2 + \cdots + w_d\, x_d + b = \boldsymbol{w}^T \boldsymbol{x} + b$$

$$\boxed{\min_{\boldsymbol{w}} \sum_{i=1}^{m} \max\{0, 1 - y^{(i)}(\boldsymbol{w}^T \boldsymbol{x}^{(i)} + b)\}}$$

❖ Hinge loss *regularized* optimization problem: $\min_{\boldsymbol{w}} \sum_{i=1}^{m} \max\{0, 1 - y^{(i)}(\boldsymbol{w}^T \boldsymbol{x}^{(i)} + b)\} + \lambda \sum_{k=1}^{d} w_k^2$

Hinge loss optimization ↔ Soft-margin SVM



$$\xi_i = \max\{0, 1 - y^{(i)}(\boldsymbol{w}^T\boldsymbol{x}^{(i)} + b)\}$$

Soft-margin SVM ⟺ **Regularized hinge loss optimization**

$$\min_{\boldsymbol{w},b,\xi} \ \boldsymbol{w}^T \cdot \boldsymbol{w} + p\sum_{j=1}^{m}\xi_j$$

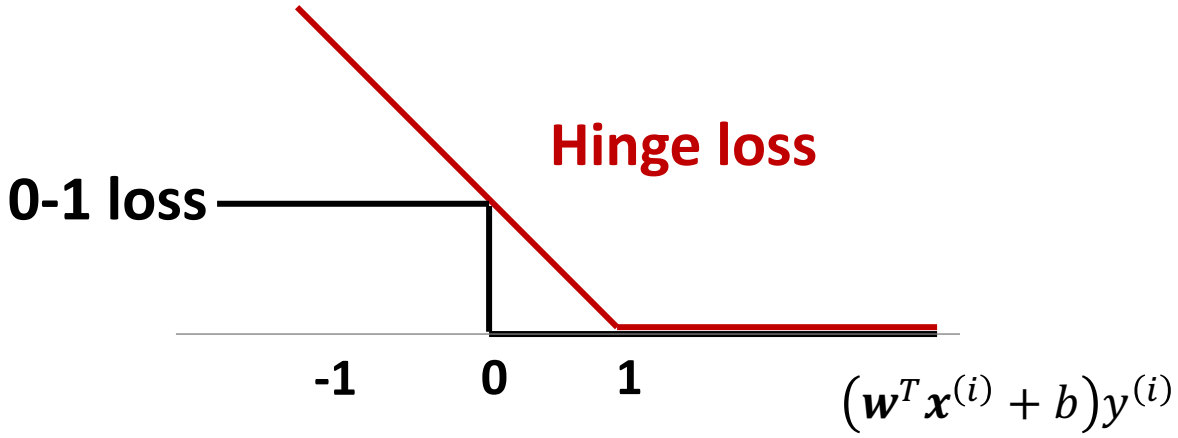
$$s.t. \quad y^{(i)}(\boldsymbol{w}^T\boldsymbol{x}^{(i)} + b) \geq 1 - \xi_i \quad i = 1, \cdots, m$$

$$\xi_i \geq 0 \quad i = 1, \cdots, m$$

$$\min_{\boldsymbol{w}} \sum_{i=1}^{m} \max\{0, 1 - y^{(i)}(\boldsymbol{w}^T\boldsymbol{x}^{(i)} + b)\} + \lambda\sum_{k=1}^{d} w_k^2$$

Hinge Loss Function

$$\min_{\boldsymbol{w}} \sum_{i=1}^{m} \max\{0, 1 - y^{(i)}(\boldsymbol{w}^T \boldsymbol{x}^{(i)} + b)\} + \lambda \sum_{k=1}^{d} w_k^2$$



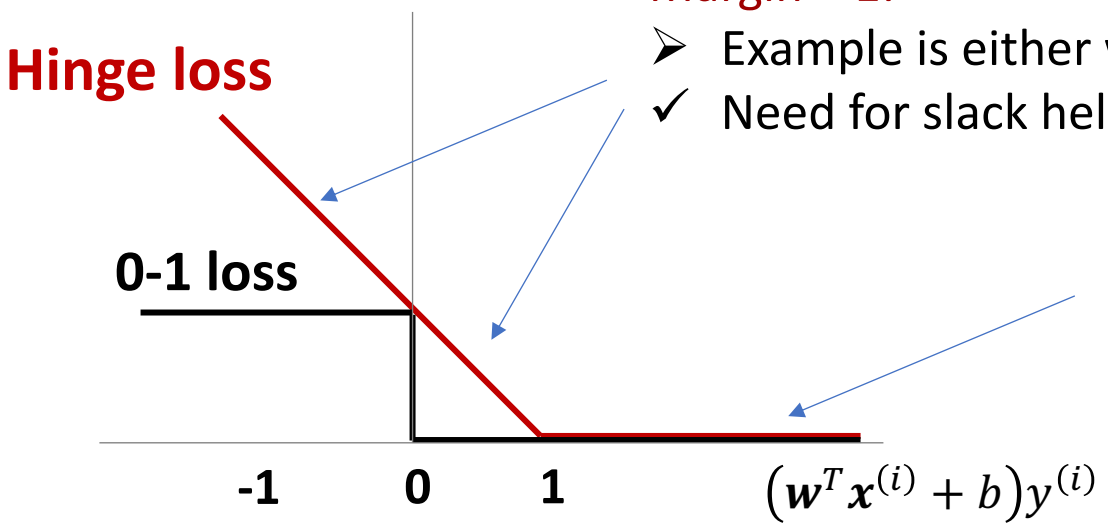
- An error signal is triggered even when there's no error (margin > 0), pushing the algorithm to keep learning until a minimal margin of at least 1 is achieved for all data
- The *regularization function* is to keep the hypothesis simple by avoiding large values for the learning parameters, favoring generalization, which traduces to *maximize margin!*
  Large margins ⇒ Good generalization
- Geometric interpretation: Zero loss needs a margin $(\boldsymbol{w}^T \boldsymbol{x})y \geq 1$, in terms of geometric margin this means: $\|w\|^{-1}(\boldsymbol{w}^T \boldsymbol{x})y \geq \|\boldsymbol{w}\|^{-1} \Longrightarrow$ Keeping $\|\boldsymbol{w}\|$ small increases geometric margin

Soft-margin SVM and Hinge loss

$$y^{(i)}\left(\boldsymbol{w}^T\boldsymbol{x}^{(i)}+b\right) \geq 1-\xi_i \quad i=1,\cdots,m$$
$$\xi_i \geq 0 \quad i=1,\cdots,m$$

$$\leftrightarrow \quad \xi_i \geq 1-\overbrace{y^{(i)}\left(\boldsymbol{w}^T\boldsymbol{x}^{(i)}+b\right)}^{\text{Margin}} \quad i=1,\cdots,m$$
$$\xi_i \geq 0$$

$$\Rightarrow \quad \xi_i(margin) = \max\left\{1-\left(\boldsymbol{w}^T\boldsymbol{x}^{(i)}+b\right)y^{(i)}, 0\right\}$$



Margin < 1:

- ➢ Example is either within margins or is incorrectly classified
- ✓ Need for slack help: increase linearly with distance from margins

Margin ≥ 1:

- ✓ Example is correctly classified
- ✓ No need for slack help!

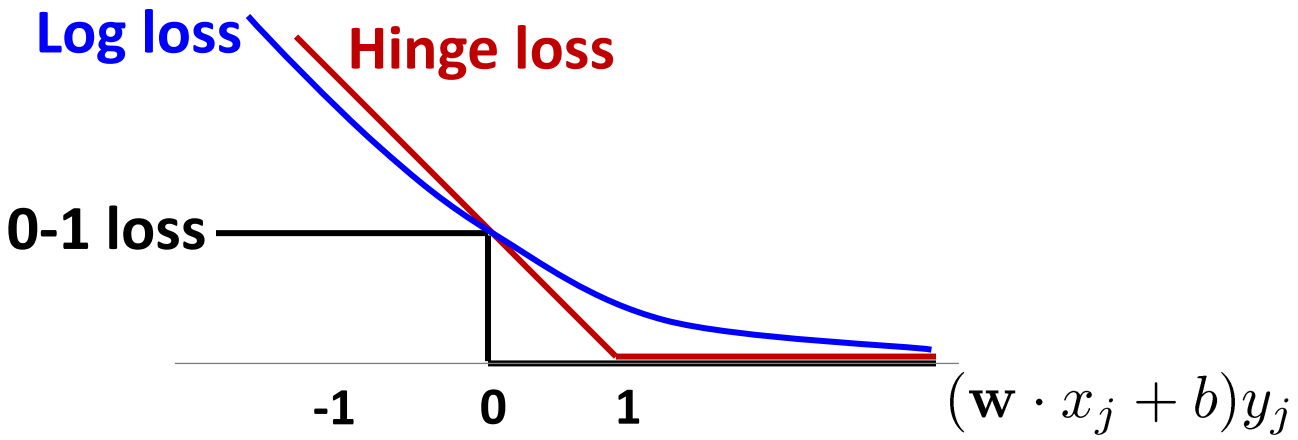
SVM vs. Logistic Regression

SVM : **Hinge loss**

$$\text{loss}(f(x_j), y_j) = (1 - (\mathbf{w} \cdot x_j + b)y_j))_+$$

Logistic Regression : **Log loss** (log conditional likelihood)

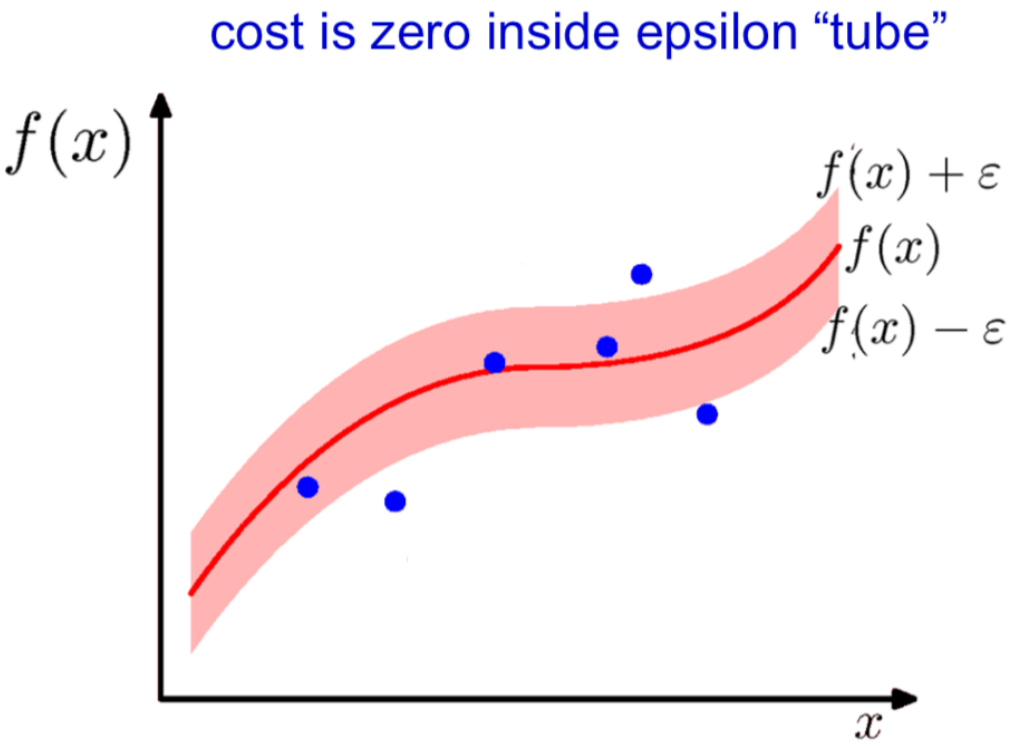
$$\text{loss}(f(x_j), y_j) = -\log P(y_j \mid x_j, \mathbf{w}, b) = \log(1 + e^{-(\mathbf{w} \cdot x_j + b)y_j})$$



- The following slides briefly illustrate how the idea of support vector machines can be extended to **regression problems**.

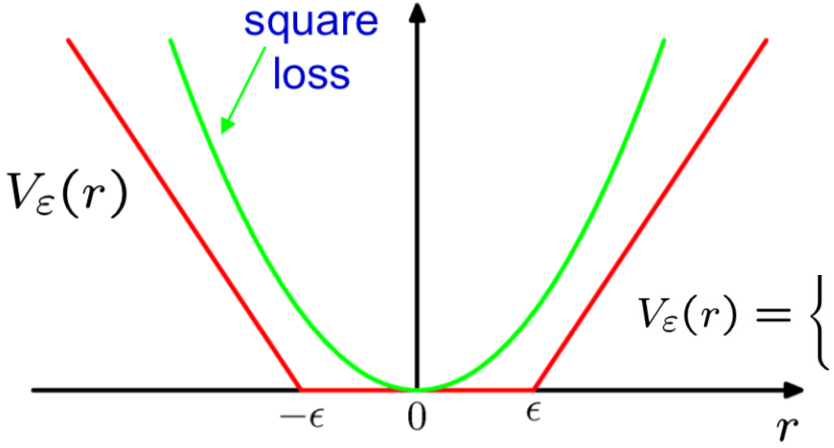
  In this setting, the notion of a margin used in classification is replaced by an ε-insensitive "tube" that surrounds the regression function, within which prediction errors are not penalized.

Support Vector Regression (SVR)



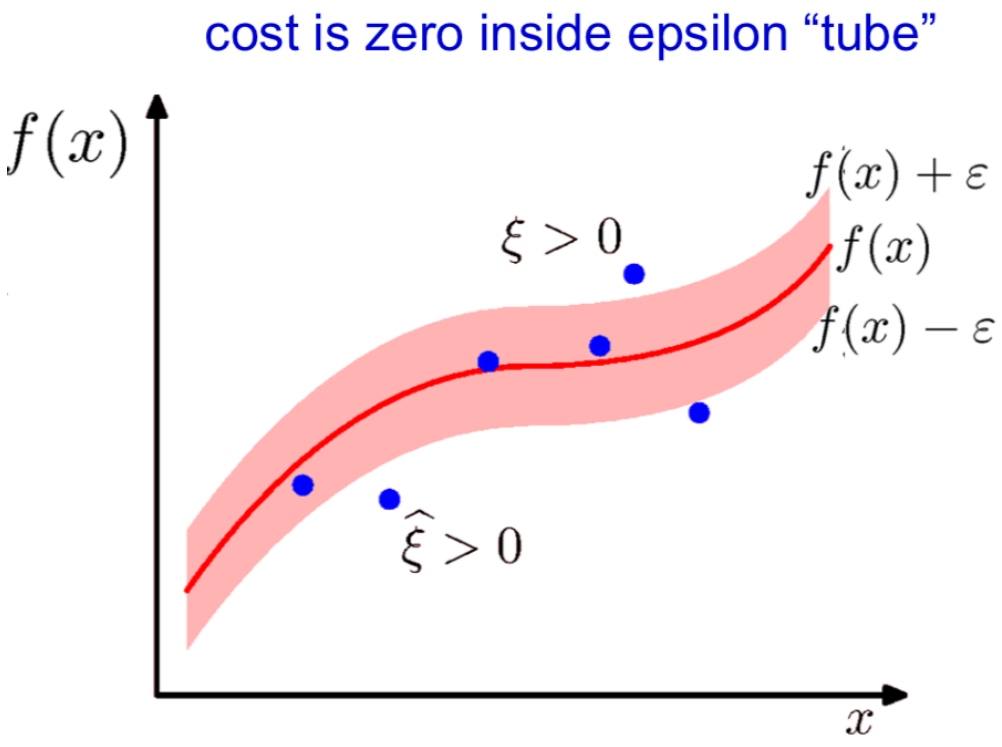
- Find the function $\hat{f}(x)$ such that the **number of points falling within the *tube* of width ε is maximized**
- Pay a **penalty** for being outside of the tube
  → Slack variables ξ like in soft-margin SVM
- **Within the tube, the loss is zero**: ε-insensitive error

$$\widetilde{E}(\mathbf{w}) = \sum_{i=1}^{N} \underbrace{V_\varepsilon\left(f(x_i, \mathbf{w}) - y_i\right)}_{\text{loss function}} + \underbrace{\frac{\lambda}{2}\|\mathbf{w}\|^2}_{\text{regularization}}$$



$$V_\varepsilon(r) = \begin{cases} 0 & \text{if } |r| \le \varepsilon \\ |r| - \varepsilon & \text{otherwise.} \end{cases}$$

Support Vector Regression (SVR)



- For each data point, x_i, two slack variables, $\xi_i, \hat{\xi}_i$ are required, depending on whether $f(x_i)$ is above or below the tube

SVR primal optimization problem:

$$\min_{\mathbf{w}\in\mathbb{R}^d,\ \xi_i,\ \hat{\xi}_i} C\sum_i^N \left(\xi_i + \hat{\xi}_i\right) + \frac{1}{2}||\mathbf{w}||^2$$

subject to

$$y_i \le f(x_i, \mathbf{w}) + \varepsilon + \xi_i, \quad y_i \ge f(x_i, \mathbf{w}) - \varepsilon - \hat{\xi}_i$$

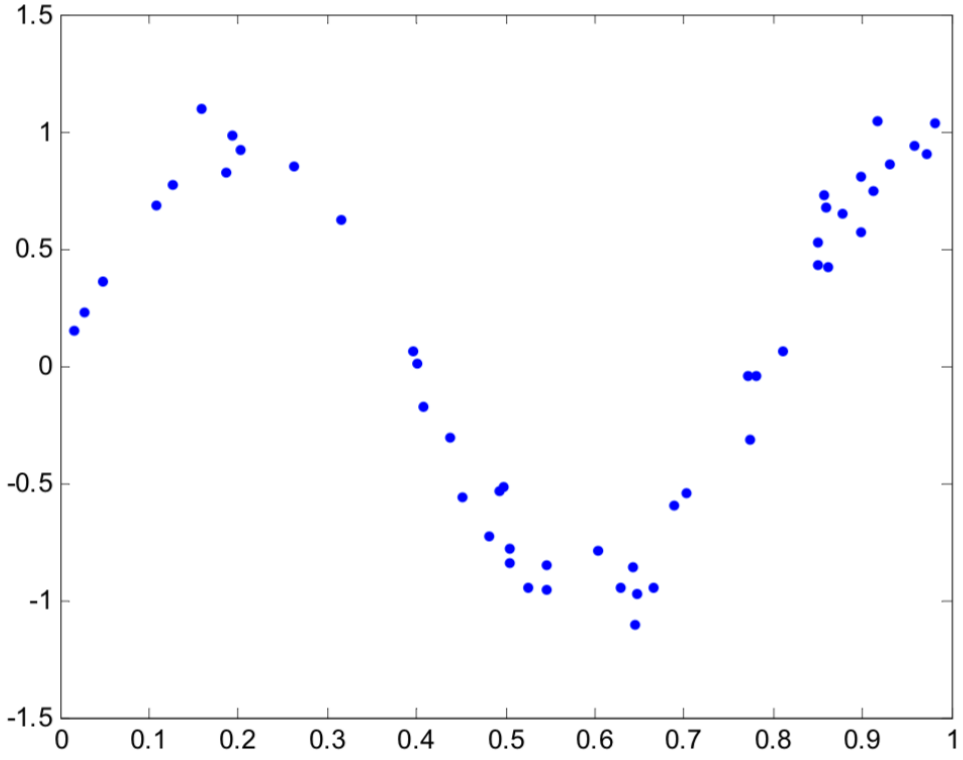
$$\xi_i \ge 0, \quad \hat{\xi}_i \ge 0 \ \text{ for } i = 1 \ldots N$$

- ✓ Quadratic optimization problem
- ○ Some points will become *support vectors*, identifying the tube
- ✓ It can be dualized and kernelized

Support Vector Regression (SVR)

Example: SV regression with Gaussian basis functions (as kernel)

- Regression function – Gaussians centred on data points
- Parameters are: C, epsilon, sigma



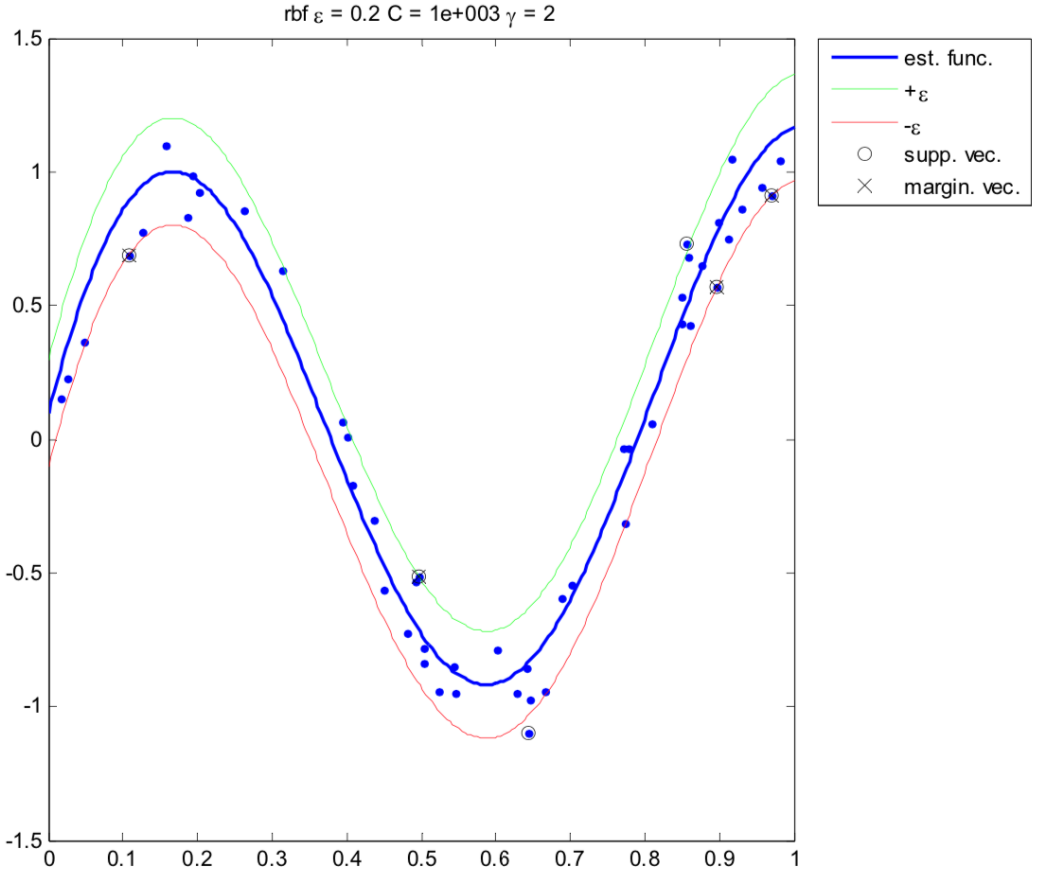
$$f(x, \mathbf{a}) = \sum_{i=1}^{N} a_i e^{-(x-x_i)^2/2\sigma^2}$$

N = 50

Support Vector Regression (SVR)

C = 1000, epsilon = 0.2, sigma = 0.5

Example: SV regression with Gaussian basis functions (as kernel)



As epsilon increases:

- fit becomes looser
- less data points are support vectors